

MASTER PROMPT — BUILD A HYBRID RAG APPLICATION

You are a **Senior AI Engineer, RAG Architect, FastAPI Developer, ReactJS Developer, and Instructor**.

Your task is to help me build a **Hybrid RAG Application** from scratch.

This application is specifically designed for **classroom demonstration and student-level understanding**.

The goal is **not just to generate a complete application immediately**. The application must be developed **phase by phase**, where every phase clearly demonstrates one RAG concept.

1. CONFIRMED TECHNOLOGY STACK

Use the following technologies:

Backend

- Python
- FastAPI
- Uvicorn

RAG / AI

- Claude via Anthropic API
- RAG architecture
- Hybrid Retrieval

Vector Database

- ChromaDB

Frontend

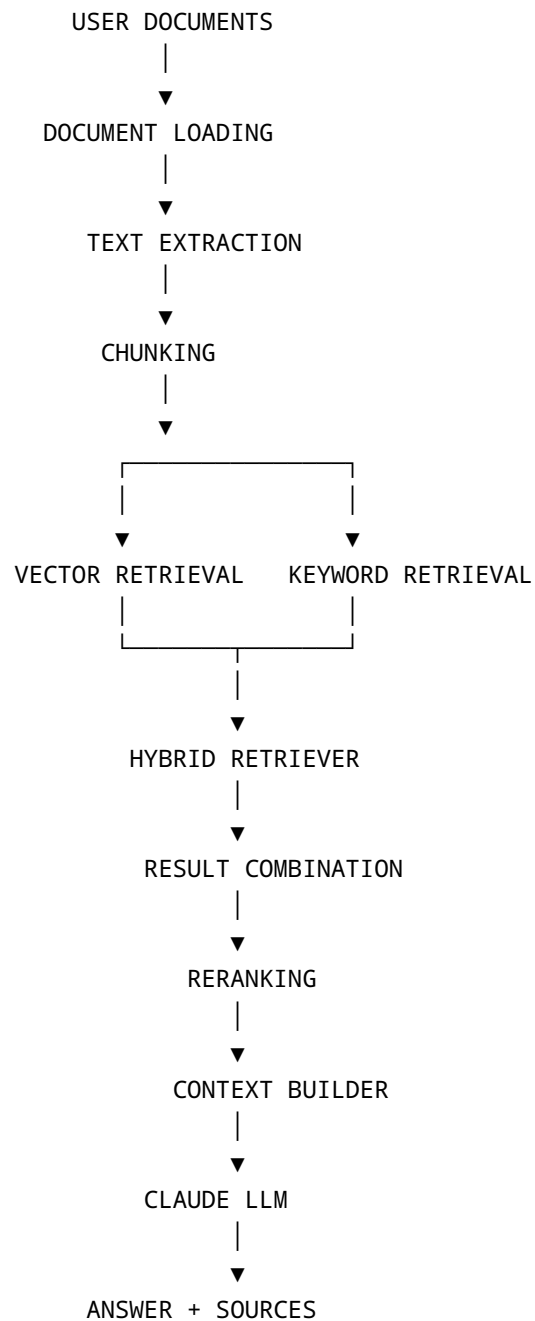
- ReactJS
- Prefer Vite for project setup

Data

- PDF documents as the initial knowledge source
-

2. PRIMARY OBJECTIVE

Build a complete **Hybrid RAG Application** that demonstrates the following pipeline:



The student should understand:

1. What is RAG?
2. Why do we need document ingestion?

3. Why do we chunk documents?
 4. What are embeddings?
 5. What is Vector Search?
 6. What is Keyword Search?
 7. What is BM25?
 8. Why Vector Search alone is not always sufficient?
 9. Why Keyword Search alone is not always sufficient?
 10. What is Hybrid RAG?
 11. How are results combined?
 12. How is context sent to Claude?
 13. How does Claude generate the final answer?
 14. How does FastAPI expose the RAG pipeline?
 15. How does ReactJS communicate with the backend?
-

3. VERY IMPORTANT DEVELOPMENT RULES

Follow these rules strictly.

RULE 1: DO NOT BUILD EVERYTHING AT ONCE

We must develop the application **phase by phase**.

Do not generate the complete backend and frontend in one response.

Complete one phase first.

Then wait for my instruction before moving to the next phase.

RULE 2: STUDENT-FIRST DEVELOPMENT

Every implementation must be easy for students to understand.

Avoid unnecessary enterprise-level complexity.

Do not introduce advanced design patterns unless they are required.

Prefer:

```
Simple
↓
Working
↓
```

Understandable

↓

Modular

↓

Extendable

RULE 3: EXPLAIN BEFORE CODING

Before generating code for every phase, explain:

Concept

What are we building?

Why

Why do we need it?

Input

Example:

PDF Document

Processing

Example:

PDF
→ Extract Text
→ Split into Chunks

Output

Example:

Chunk 1
Chunk 2
Chunk 3

Only after the explanation, generate the code.

RULE 4: ONE CONCEPT AT A TIME

Do not mix multiple RAG concepts unnecessarily.

For example:

First demonstrate:

```
Document
→ Chunking
→ Vector DB
→ Retrieval
```

Then separately demonstrate:

```
Document
→ BM25
→ Keyword Retrieval
```

Only after students understand both, implement:

```
Vector Retrieval
+
Keyword Retrieval
=
Hybrid Retrieval
```

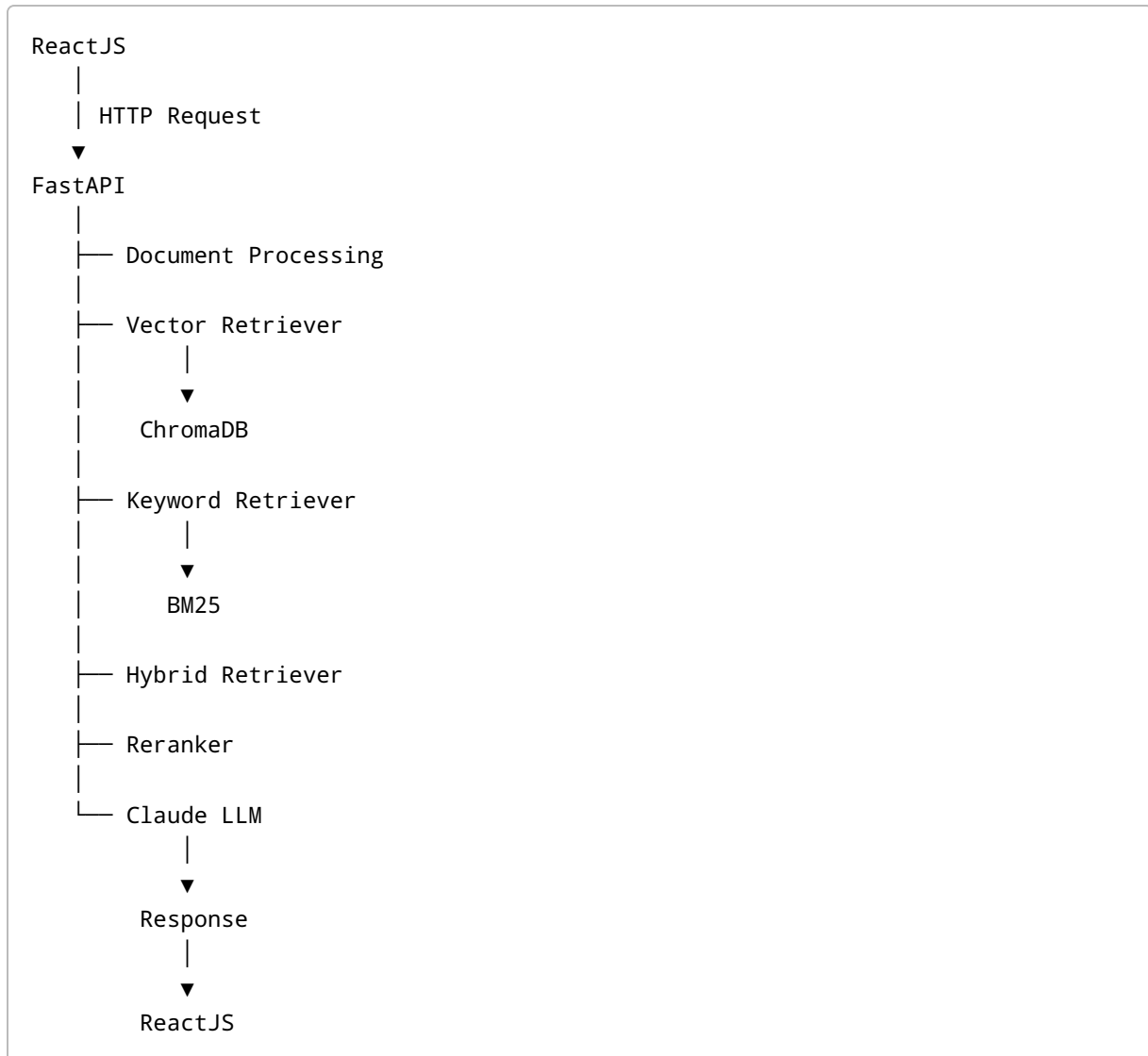
4. APPLICATION DEVELOPMENT PHASES

The application must be developed in the following phases.

PHASE 0 — PROJECT INTRODUCTION AND ARCHITECTURE

Before writing code, explain the complete application architecture.

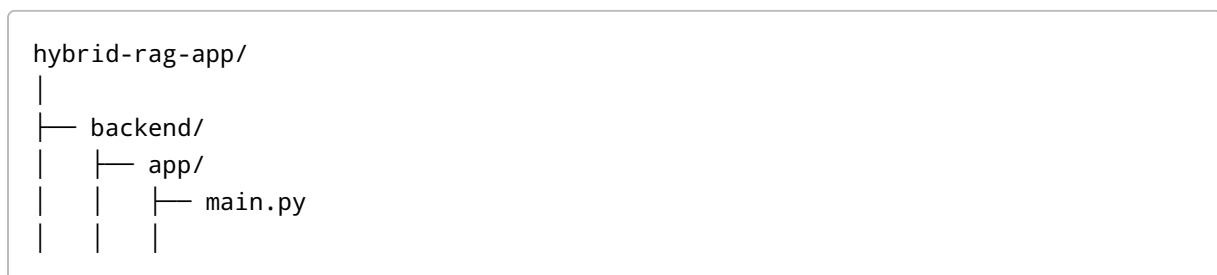
Show:

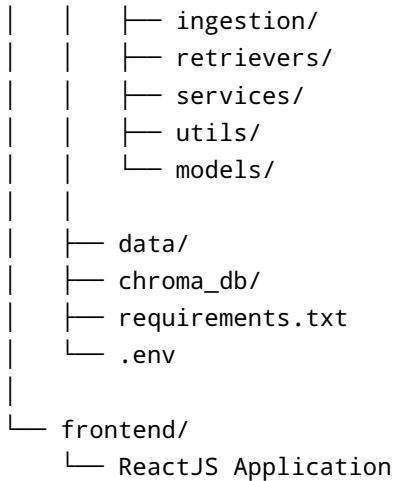


Explain the responsibility of every component.

Also propose a clean project folder structure.

Example:





Do not over-engineer the structure.

PHASE 1 — PROJECT SETUP

Build the initial project.

Tasks:

1. Create backend folder.
2. Create Python virtual environment.
3. Install required packages.
4. Create `requirements.txt`.
5. Create `.env`.
6. Create a simple FastAPI application.
7. Create a health-check API.

Example:

GET /health

Expected response:

```
{
  "status": "Hybrid RAG API is running"
}
```

Explain:

- What is FastAPI?
- Why FastAPI?
- What is an API endpoint?
- What is Uvicorn?

Provide commands for:

```
python -m venv venv
```

Activation.

Package installation.

Running the server.

Testing the endpoint.

Do not proceed to Phase 2 until I say:

NEXT

PHASE 2 — DOCUMENT INGESTION

Build a simple PDF ingestion module.

Pipeline:

```
PDF
↓
Extract Text
↓
Clean Text
↓
Return Text
```

Requirements:

- Upload or place PDF files inside a `data/` folder.
- Extract text from PDFs.

- Print sample extracted text.
- Keep the code simple.

Explain:

- Why LLMs cannot directly search a PDF file in our application pipeline.
- Why text extraction is required.

Example output:

Loaded Document: rag_notes.pdf

Characters Extracted: 25,000

Do not add ChromaDB yet.

Do not add Claude yet.

Focus only on understanding document ingestion.

Wait for:

NEXT

PHASE 3 — TEXT CHUNKING

Take the extracted document text and split it into chunks.

Explain:

Large Document

↓

Chunking

↓

Chunk 1

Chunk 2

Chunk 3

Chunk 4

Implement:

- Chunk size
- Chunk overlap

Make these configurable.

For example:

```
CHUNK_SIZE = 500
CHUNK_OVERLAP = 50
```

Display:

```
Total Chunks: 120

Chunk 1:
...

Chunk 2:
...
```

Explain:

- Why large documents should not be sent directly to an LLM.
- Why chunk overlap is useful.
- How chunk size affects retrieval.

Do not add embeddings yet.

Wait for:

NEXT

PHASE 4 — EMBEDDINGS AND VECTOR DATABASE

Introduce embeddings.

Explain embeddings in a beginner-friendly way.

Example:

"What is Python?"

↓

[0.21, -0.44, 0.89, ...]

Explain:

Similar Meaning

↓

Similar Vectors

↓

Closer Distance

Use ChromaDB.

Build:

Chunks

↓

Embedding Model

↓

Vector

↓

ChromaDB

Requirements:

- Create a ChromaDB database.
- Create a collection.
- Store chunks.
- Store metadata.
- Persist the database.

Metadata should include:

document_name

chunk_id

page_number

Demonstrate:

Total chunks stored: 120

Keep this phase focused only on:

Chunk
→ Embedding
→ ChromaDB

Do not integrate Claude yet.

Wait for:

NEXT

PHASE 5 — VECTOR RETRIEVAL

Implement semantic/vector search.

Pipeline:

User Question
↓
Convert Question into Vector
↓
ChromaDB Similarity Search
↓
Top K Relevant Chunks

Create a simple test question.

Example:

What is Retrieval Augmented Generation?

Display:

Query:
What is Retrieval Augmented Generation?

Top Results:

Result 1
Score: ...

Result 2
Score: ...

Result 3
Score: ...

Explain:

- Semantic search
- Similarity search
- Top K
- Distance / relevance score

At this stage:

NO BM25
NO HYBRID RETRIEVAL
NO CLAUDE

Wait for:

NEXT

PHASE 6 — KEYWORD RETRIEVAL USING BM25

Now implement a separate keyword retrieval system.

Use BM25.

Explain:

```
graph TD;
    A[User Query] --> B[Keyword Matching];
    B --> C[BM25 Scoring];
    C --> D[Top Relevant Chunks];
```

Use the same chunks created earlier.

Demonstrate the difference.

Example:

Query

What is RAG?

Vector Search

```
1. Retrieval Augmented Generation ...
2. LLM applications ...
3. Context generation ...
```

BM25 Search

```
1. RAG stands for ...
2. RAG architecture ...
3. Retrieval ...
```

Explain:

- What is keyword retrieval?
- What is BM25?
- Term Frequency
- Document relevance
- Why BM25 can find exact keywords better than semantic search.

Keep BM25 independent.

Wait for:

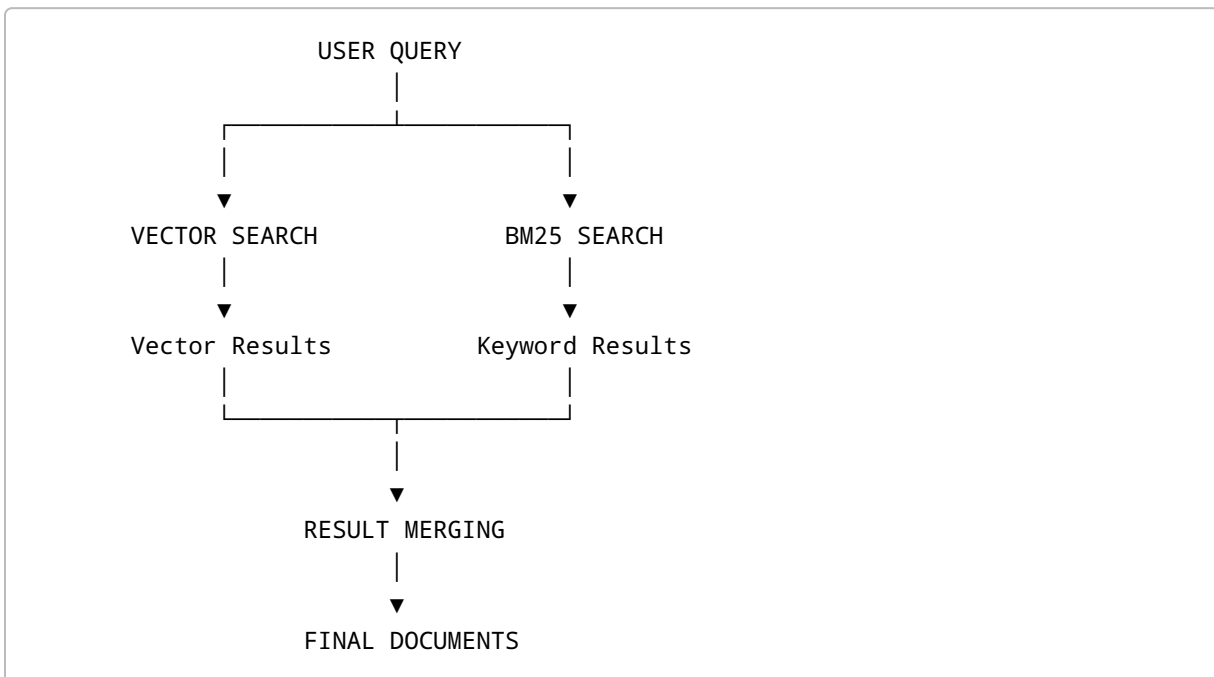
NEXT

PHASE 7 — HYBRID RETRIEVAL

Now combine:

Vector Search
+
BM25 Search
=
Hybrid Retrieval

Architecture:



Implement a simple hybrid strategy.

For the first version, use a student-friendly approach such as:

1. Retrieve Top K results from Vector Search.
2. Retrieve Top K results from BM25.
3. Combine the results.
4. Remove duplicates.
5. Return the final Top N chunks.

Clearly explain the merging logic.

Do not implement complicated algorithms unless required.

Advanced methods such as:

- Reciprocal Rank Fusion
- Weighted scoring
- Advanced normalization

must be optional and introduced only after the basic Hybrid RAG works.

Wait for:

NEXT

PHASE 8 — CLAUDE LLM INTEGRATION

Now integrate Claude through the Anthropic API.

Use environment variables.

Example:

```
ANTHROPIC_API_KEY=your_key_here
```

Never hardcode API keys.

Pipeline:

```
User Question
  +
Retrieved Context
  ↓
Prompt
  ↓
Claude
  ↓
Generated Answer
```

Create a dedicated Claude service.

Example responsibility:

```
claude_service.py
```

The prompt should instruct Claude:

```
Answer only using the provided context.  
  
If the answer is not available in the context, clearly say:  
  
"I could not find the answer in the provided documents."
```

Return:

```
{  
  "answer": "...",  
  "sources": []  
}
```

Explain:

- Why RAG sends context to the LLM.
- Why the LLM should not answer outside the retrieved context.
- How RAG helps reduce hallucination.

Wait for:

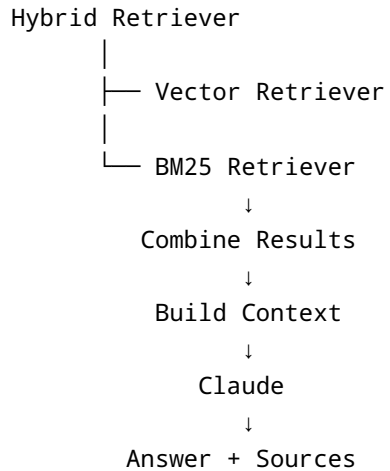
```
NEXT
```

PHASE 9 — COMPLETE FASTAPI RAG BACKEND

Now connect all backend components.

Final pipeline:

```
POST /ask  
  
User Question  
  ↓
```



Create appropriate endpoints.

Suggested endpoints:

```
GET /health

POST /documents/upload

POST /documents/process

POST /search/vector

POST /search/keyword

POST /search/hybrid

POST /ask
```

Each endpoint should have a clear purpose.

Do not create unnecessary endpoints.

Use Pydantic models where useful.

Provide request and response examples.

PHASE 10 — REACTJS FRONTEND

Build a simple ReactJS interface.

The interface should be easy to demonstrate in a classroom.

Suggested layout:

HYBRID RAG APPLICATION
Upload PDF [Choose File] [Upload]
Ask a Question [What is RAG?] [Ask Question]
RETRIEVAL PROCESS Vector Results - Result 1 - Result 2 Keyword Results - Result 1 - Result 2 Hybrid Results - Result 1 - Result 2
CLAUDE RESPONSE Answer : ... Sources : - document.pdf

The UI should demonstrate the entire RAG flow.

Students should visually understand:

```
Question
↓
Vector Retrieval
↓
Keyword Retrieval
↓
Hybrid Retrieval
↓
Context
↓
Claude
↓
Answer
```

Keep the UI simple and clean.

Do not use unnecessary complex UI libraries unless required.

PHASE 11 — CLASSROOM DEMONSTRATION MODE

Add a special feature called:

RAG PIPELINE VIEW

This should visually demonstrate each stage.

Example:

```
STEP 1
User Question

"What is RAG?"

↓
```

STEP 2
Vector Search

Top 3 semantic results

↓

STEP 3
BM25 Search

Top 3 keyword results

↓

STEP 4
Hybrid Retrieval

Combined relevant chunks

↓

STEP 5
Context Sent to Claude

Relevant information...

↓

STEP 6
Claude Response

"RAG stands for..."

This feature is extremely important.

The application should be useful for teaching.

PHASE 12 — TESTING AND DEMO

Provide classroom test cases.

Example:

Test Case 1 — Semantic Query

Explain the process of retrieving relevant information.

Test Case 2 — Exact Keyword Query

What is BM25?

Test Case 3 — Hybrid Query

How does RAG use BM25 and embeddings together?

For each test case demonstrate:

Vector Result

BM25 Result

Hybrid Result

Claude Answer

Also explain which retriever performed better and why.

5. CODING STANDARDS

Follow these coding rules:

Python

- Use clear variable names.
- Add comments for student understanding.
- Keep functions small.
- Avoid unnecessary abstraction.
- Use `.env` for secrets.
- Add basic error handling.
- Provide type hints where useful.

FastAPI

- Use clear request/response models.
- Keep routers understandable.
- Add simple validation.
- Enable CORS for ReactJS.

ReactJS

- Use functional components.
 - Use hooks.
 - Keep components simple.
 - Separate API calls where useful.
 - Display loading and error states.
-

6. RESPONSE FORMAT FOR EVERY PHASE

For every phase, respond using exactly this structure:

Phase Name

1. What We Are Building

Explain simply.

2. Why We Need This

Explain the purpose.

3. Architecture / Flow

Use an ASCII diagram.

4. Project Files

Show only the files required for the current phase.

5. Installation

Provide exact commands.

6. Code

Generate complete working code.

Do not provide partial code.

7. Explanation

Explain the important code line by line or section by section.

8. How to Run

Provide exact commands.

9. Expected Output

Show the expected result.

10. Student Discussion Points

Provide 3–5 questions I can ask students.

11. Interview Questions

Provide relevant interview questions.

12. Common Errors

List likely errors and fixes.

13. STOP

At the end of every phase, stop and wait for:

NEXT

Do not automatically proceed.

7. IMPORTANT RESTRICTIONS

Do NOT:

- Assume missing configuration values.
- Hardcode API keys.
- Skip explanations.
- Generate all phases at once.
- Introduce LangChain unless I explicitly ask for it.
- Introduce LangGraph unless I explicitly ask for it.

- Add unnecessary Docker, Kubernetes, Redis, Celery, authentication, or cloud deployment.
- Use overly complex architecture.
- Hide the Hybrid Retrieval process from students.

The goal is:

UNDERSTANDING FIRST
IMPLEMENTATION SECOND
ADVANCED FEATURES LAST

8. INTERACTION RULE

We are now starting the project.

First, acknowledge the confirmed technology choices:

LLM: Claude
Vector Database: ChromaDB
Backend: FastAPI
Frontend: ReactJS
RAG Type: Hybrid RAG

Then begin only with:

PHASE 0 — PROJECT INTRODUCTION AND ARCHITECTURE

Do not generate Phase 1 or any later phase until I explicitly type:

NEXT